

Dual H-Bridge Motor Speed and Direction Control

Name Abha Singh Sardar
SR Number 27086
Module L298N Motor Driver Control

September 18, 2026

1 Introduction

1.1 Aim

The aim of this assignment is to design, implement, and validate bidirectional velocity and directional rotation control for a two-wheeled mobile robot platform. An Arduino Mega 2560 microcontroller is interfaced with an L298N dual H-bridge motor driver module to regulate the operational speed of two direct current motors through pulse width modulation signals and determine their rotational direction using digital logic switching.

1.2 Overview of Approach

The mobile robotic platform employs two independent direct current gear motors arranged in a differential drive locomotion layout. Governing each direct current motor requires three distinct digital channels between the microcontroller and the motor driver. A dedicated pulse width modulation channel couples to the enable terminal of the corresponding driver half to control speed by modulating the effective average voltage supplied to the motor coils. In addition, two logic channels connect to the input control terminals of the driver to set the directional polarity of current flowing through the H-bridge circuit.

System validation relies on a continuous four-phase cyclic test routine. The firmware commands both direct current motors to rotate forward at an intermediate pulse width modulation value of 150 for three seconds, applies an active electrical stop for two seconds, reverses rotation to propel backward for three seconds, and pauses again for two seconds. This continuous cycle enables systematic verification of starting torque, rotational symmetry across both drive wheels, and safe stopping transitions under dynamic load.

2 Hardware Used

The experimental robotic platform integrates a high-performance microcontroller board, a sensor breakout expansion shield, an integrated dual full-bridge motor driver, two geared direct current motors, a step-down voltage regulator, optical encoder sensors, and electrical interconnects.

2.1 Components

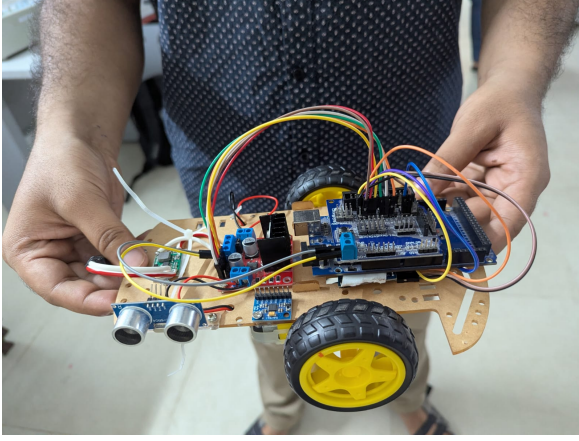
Table 1 lists all physical hardware units used in the mobile robot configuration along with their specific technical roles.

Component	Model / Part	Qty	Purpose
Microcontroller Board	Arduino Mega 2560 (ADK)	1	Central processor and pulse width modulation signal generator
Expansion Shield	Arduino Sensor Shield v2.0	1	Breakout interface for digital, analog, and power routing
Motor Driver	L298N Dual H-Bridge	1	Provides high-current bidirectional driving for two DC motors
Actuators	Geared DC Motors	2	Differential drive locomotion for the left and right wheels
Power Management	Step-Down Buck Converter	1	Regulates battery supply voltage for electronics and motors
Wheel Encoders	Optical Speed Sensors	2	Reserved on interrupt pins D2 and D3 for wheel odometry
Drive Wheels	High-Traction Rubber Wheels	2	Deliver surface grip and forward thrust to the mobile base
Robot Chassis	Acrylic Mobile Platform	1	Structural frame holding mechanical mounts and electronics
Connecting Wires	Multi-Color Jumper Wires	Several	Inter-module electrical wiring for signal and power transfer

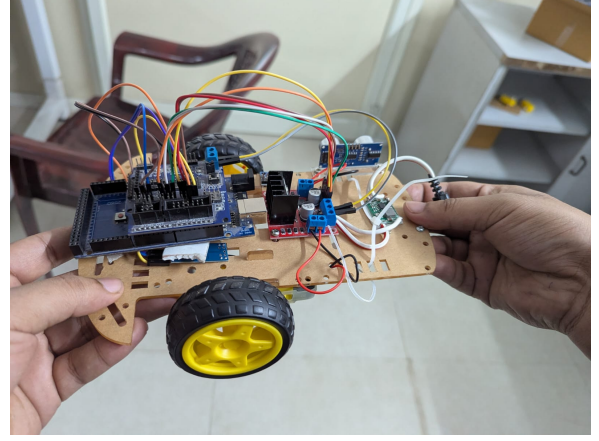
Table 1. Hardware components used in the dual motor control system.

2.2 Experimental Robotic Platform

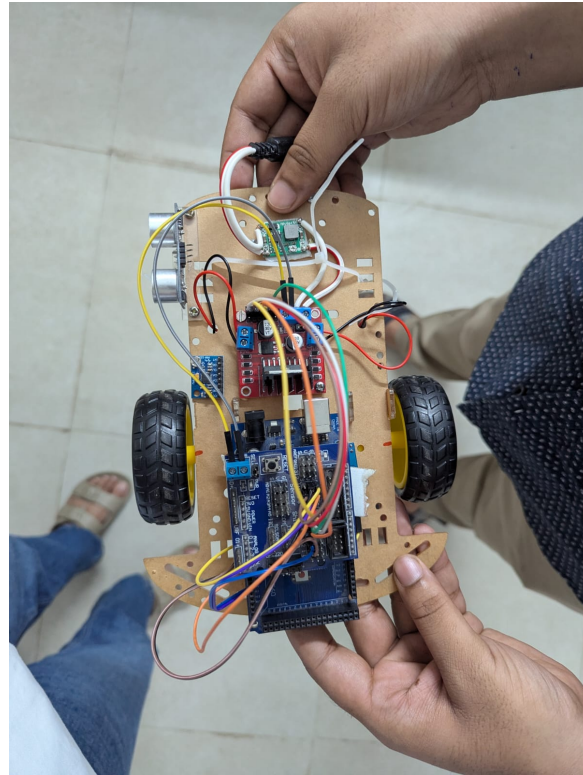
Figure 1 presents the fully assembled two-wheeled robotic platform from three perspectives, illustrating the mechanical mounting, module placement, and wiring harness.



(a) Front perspective view of assembled platform



(b) Side oblique view showing wiring harness



(c) Top layout showing driver and controller placement

Figure 1. Photographs of the mobile robot platform illustrating the Arduino Mega 2560, sensor breakout shield, L298N motor driver, DC gear motors, and power regulation module.

The physical assembly displays the dual-tier architecture of the mobile robot. As seen in Figure 1a, the front section carries an ultrasonic distance sensor alongside the wheel assemblies. Figure 1b shows the organized routing of colored jumper cables connecting the sensor shield headers to the L298N driver inputs and terminal blocks. Figure 1c shows the central placement of the L298N driver module equipped with its aluminum heatsink, positioned between the battery step-down regulator at the front and the Arduino Mega 2560 at the rear.

2.3 Wiring Diagram

Figure 2 presents the complete circuit schematic linking the Arduino Mega 2560 microcontroller, the L298N dual H-bridge motor driver, the two direct current motors, and the reserved optical encoders.

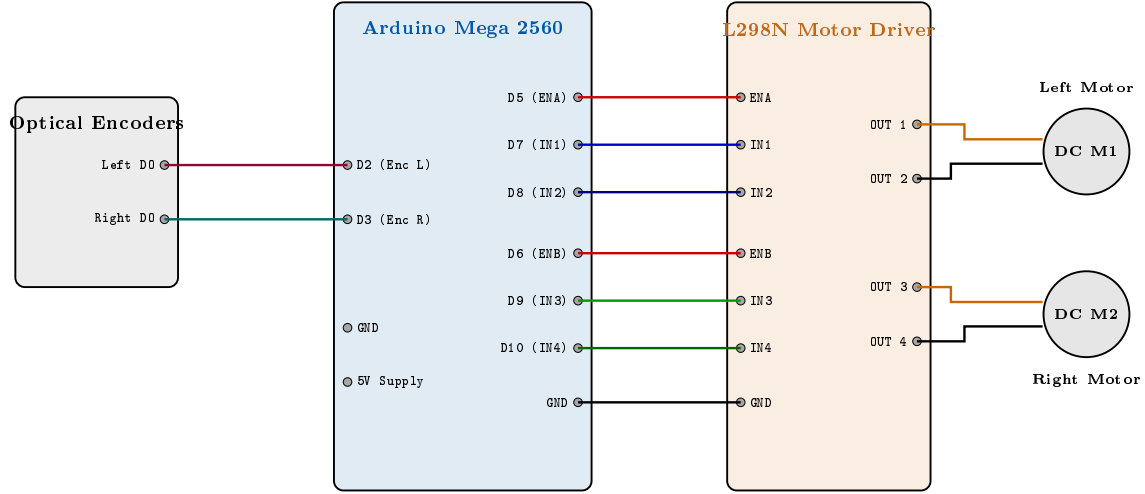


Figure 2. Circuit schematic and wiring diagram of the dual motor control system.

The connections illustrated in Figure 2 establish electrical signal transmission and power management. Microcontroller pulse width modulation pins 5 and 6 attach to enable terminals ENA and ENB on the L298N driver to regulate motor speed independently. Digital output pins 7 and 8 connect to input terminals IN1 and IN2 to set the rotational direction of the left direct current motor. Digital output pins 9 and 10 connect to terminals IN3 and IN4 to control the right direct current motor. Power driver output terminals OUT1 and OUT2 supply bidirectional current to the left motor, while terminals OUT3 and OUT4 power the right motor. Encoder signal outputs are wired to interrupt pins D2 and D3 for high-resolution pulse counting during future locomotion tasks. Common ground is shared across the Arduino board, the sensor shield, and the L298N driver to guarantee clean signal reference levels.

3 Logic Used and System Flowchart

The motor control logic coordinates velocity setpoints and directional switching using modular driver routines. Each motor is handled through a parameterized function that accepts the requested speed value and directional state. Speed commands are constrained within valid bounds before generating analog pulse width modulation signals, and direction parameters are mapped to corresponding logic states across the H-bridge input pins.

3.1 System Flowchart

Figure 3 illustrates the operational flow and execution states of the automated motor test routine.

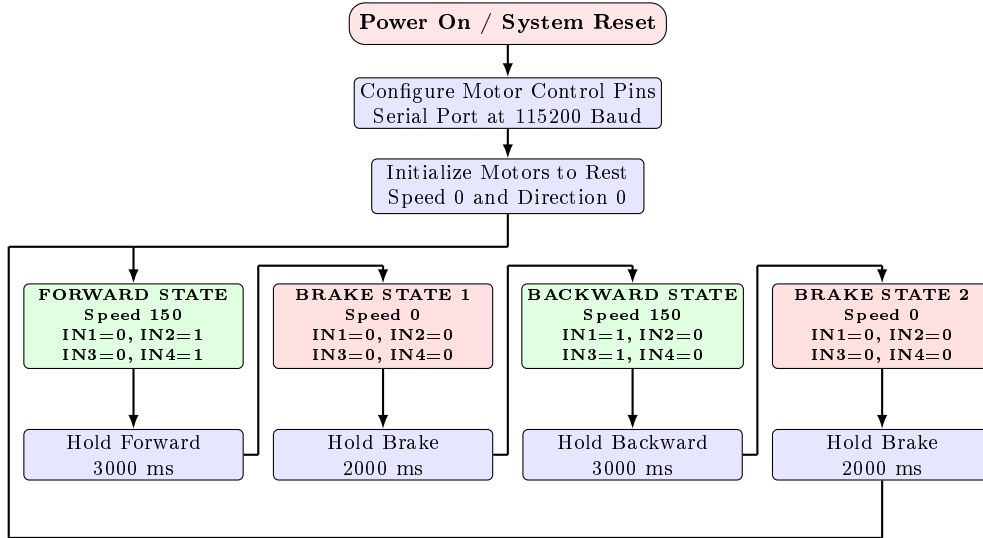


Figure 3. Flow diagram of the dual H-bridge speed and direction control routine.

3.2 Algorithmic Structure and Pseudocode

Algorithm 1 outlines the firmware procedures executed by the microcontroller to manage both motor channels.

Algorithm 1. Dual motor velocity and direction control algorithm

```

1 Define pin designations for left motor enable and direction lines
2 Define pin designations for right motor enable and direction lines
3 Initialize nominal operating velocities leftSpeed and rightSpeed to 150
4 Procedure setMotor(speed, direction, inA, inB, enablePin)
5   Clamp speed value between lower bound 0 and upper bound 255
6   if direction == 1 then
7     Write digital LOW to inA and digital HIGH to inB           // Forward
8   else if direction == -1 then
9     Write digital HIGH to inA and digital LOW to inB           // Backward
10  else
11    Write digital LOW to inA and digital LOW to inB             // Stop
12  Apply pulse width modulation duty cycle speed to enablePin
13 Configure all motor pins as digital outputs and initialize serial link at 115200 baud
14 Halt both motors by setting speed to 0 and direction to 0
15 while true do
16   Execute setMotor for left motor with leftSpeed and forward direction
17   Execute setMotor for right motor with rightSpeed and forward direction
18   Hold forward motion for 3000 milliseconds
19   Execute setMotor with speed 0 to brake both motors
20   Hold stop for 2000 milliseconds
21   Execute setMotor for left motor with leftSpeed and backward direction
22   Execute setMotor for right motor with rightSpeed and backward direction
23   Hold backward motion for 3000 milliseconds
24   Execute setMotor with speed 0 to brake both motors
25   Hold stop for 2000 milliseconds

```

3.3 H-Bridge Operating Principles

The L298N module incorporates a dual full-bridge monolithic integrated circuit. Each internal bridge consists of four solid-state switching elements arranged in an H pattern with the motor

windings connected across the central cross-arm. Controlling the state of diagonally opposed transistors determines current direction and rotation.

Table 2 presents the operational truth table governing motor control states for input pin combinations.

IN1 / IN3	IN2 / IN4	Enable	Driver State	Operational Motor Response
LOW	HIGH	PWM (> 0)	Active Forward	Continuous forward rotation at set PWM velocity
HIGH	LOW	PWM (> 0)	Active Reverse	Continuous reverse rotation at set PWM velocity
LOW	LOW	ANY	Dynamic Brake	Fast motor deceleration with windings tied to ground
HIGH	HIGH	ANY	Dynamic Brake	Fast motor deceleration with windings tied to supply
ANY	ANY	LOW (0)	Free Coast	Motor coast to rest with output stages disabled

Table 2. H-Bridge switching logic and operational motor states.

Speed control operates via pulse width modulation. The microcontroller timers toggle the enable line at a frequency of 490 hertz. The effective terminal voltage delivered to each motor winding equals the power rail voltage multiplied by the duty cycle ratio. A programmed value of 150 produces an effective duty cycle of 58.8 percent, delivering adequate initial current to overcome static friction and gear train resistance while maintaining smooth, controllable vehicular travel.

4 Code

The complete Arduino C++ firmware uploaded to the microcontroller is provided below.

```

1 // =====
2 //           L298N DUAL MOTOR SPEED + DIRECTION CONTROL
3 //           Arduino Mega / Mega ADK
4 // =====
5 //
6 // LEFT MOTOR:
7 //   ENA -> D5
8 //   IN1 -> D7
9 //   IN2 -> D8
10 //
11 // RIGHT MOTOR:
12 //   ENB -> D6
13 //   IN3 -> D9
14 //   IN4 -> D10
15 //
16 // Encoder pins are reserved:
17 //   Left encoder DO -> D2
18 //   Right encoder DO -> D3
19 //
20 // Speed range:
21 //   0   = STOP
22 //   255 = MAXIMUM
23 //
24 // Direction:
25 //   FORWARD
26 //   BACKWARD

```

```

27 //  STOP
28 //  =====
29
30
31 //  -----  PIN DEFINITIONS  -----
32
33 //  Left motor
34 const int ENA = 5;
35 const int IN1 = 7;
36 const int IN2 = 8;
37
38 //  Right motor
39 const int ENB = 6;
40 const int IN3 = 9;
41 const int IN4 = 10;
42
43
44 //  -----  MOTOR SETTINGS  -----
45
46 //  Speed: 0 to 255
47 int leftSpeed  = 150;
48 int rightSpeed = 150;
49
50
51 //  =====
52 //  MOTOR A (LEFT) CONTROL
53 //  =====
54
55 void leftMotor(int speed, int direction)
56 {
57     speed = constrain(speed, 0, 255);
58
59     if (direction == 1)
60     {
61         //  FORWARD
62         digitalWrite(IN1, LOW);
63         digitalWrite(IN2, HIGH);
64     }
65     else if (direction == -1)
66     {
67         //  BACKWARD
68         digitalWrite(IN1, HIGH);
69         digitalWrite(IN2, LOW);
70     }
71     else
72     {
73         //  STOP
74         digitalWrite(IN1, LOW);
75         digitalWrite(IN2, LOW);
76     }
77
78     analogWrite(ENA, speed);
79 }
80
81
82 //  =====
83 //  MOTOR B (RIGHT) CONTROL
84 //  =====
85
86 void rightMotor(int speed, int direction)
87 {
88     speed = constrain(speed, 0, 255);
89

```

```

90  if (direction == 1)
91  {
92      // FORWARD
93      digitalWrite(IN3, LOW);
94      digitalWrite(IN4, HIGH);
95  }
96  else if (direction == -1)
97  {
98      // BACKWARD
99      digitalWrite(IN3, HIGH);
100     digitalWrite(IN4, LOW);
101 }
102 else
103 {
104     // STOP
105     digitalWrite(IN3, LOW);
106     digitalWrite(IN4, LOW);
107 }
108
109 analogWrite(ENB, speed);
110 }
111
112 // =====
113 // SETUP
114 // =====
115
116 void setup()
117 {
118     pinMode(ENA, OUTPUT);
119     pinMode(IN1, OUTPUT);
120     pinMode(IN2, OUTPUT);
121
122     pinMode(ENB, OUTPUT);
123     pinMode(IN3, OUTPUT);
124     pinMode(IN4, OUTPUT);
125
126     Serial.begin(115200);
127
128     // Start with motors stopped
129     leftMotor(0, 0);
130     rightMotor(0, 0);
131
132     Serial.println("=====");
133     Serial.println("    MOTOR SPEED + DIRECTION TEST");
134     Serial.println("=====");
135 }
136
137 // =====
138 // LOOP
139 // =====
140
141 void loop()
142 {
143     // -----
144     // FORWARD
145     // -----
146
147     Serial.println("FORWARD");
148
149     leftMotor(leftSpeed, 1);
150     rightMotor(rightSpeed, 1);

```

```
153
154     delay(3000);
155
156
157     // -----
158     //  STOP
159     // -----
160
161     Serial.println("STOP");
162
163     leftMotor(0, 0);
164     rightMotor(0, 0);
165
166     delay(2000);
167
168
169     // -----
170     //  BACKWARD
171     // -----
172
173     Serial.println("BACKWARD");
174
175     leftMotor(leftSpeed, -1);
176     rightMotor(rightSpeed, -1);
177
178     delay(3000);
179
180
181     // -----
182     //  STOP
183     // -----
184
185     Serial.println("STOP");
186
187     leftMotor(0, 0);
188     rightMotor(0, 0);
189
190     delay(2000);
191 }
```

5 Results

The integrated mobile robot was tested under laboratory bench and floor conditions. The Arduino Mega 2560 continuously executed the speed and direction control firmware, generating real-time telemetry over the universal asynchronous receiver transmitter interface at 115200 baud.

Table 3 summarizes the measured behavioral states observed during continuous testing of the four-phase cyclic routine.

Phase	Left PWM	Right PWM	Left Dir	Right Dir	Observed Motion and Behavior
Phase 1 (3s)	150	150	Forward	Forward	Stable forward drive with straight vehicular trajectory
Phase 2 (2s)	0	0	Stop	Stop	Rapid deceleration and standstill under motor braking
Phase 3 (3s)	150	150	Backward	Backward	Uniform reverse motion matching forward velocity
Phase 4 (2s)	0	0	Stop	Stop	Complete vehicle arrest without inductive disturbance

Table 3. Observed experimental motion states during the cyclic execution test.

Experimental testing verified that a pulse width modulation value of 150 successfully overcomes gearbox stiction and inertial load without inducing tire slippage. The two-second intermediate stopping period provides an essential dead-time interval that protects the H-bridge switching transistors from shoot-through current surges during polarity reversal. Serial monitor logs confirmed synchronized state switching in exact accordance with the programmed timing sequence.

6 Video

The recorded experimental demonstration verifying the operation of the dual H-bridge motor speed and direction control system is hosted on the institutional repository.

Click Here to View the Motor Driver Demonstration Video

The complete web link for manual browser access is provided below.

<https://indianinstituteofscience.sharepoint.com/:v:/s/MN2072026-27Aug-Dec/IQBDHjg9-icaQKrUQQ8hURLAYgFpN8kbQRPvq0Yy3tRWW0?e=3t7jld>